

Update 1 – Weekly Update Report

CSE 499B: Senior Design I | Section: 12 | Group: 03

Week Update Summary

We achieved major milestones in automation, model training, and component development for this project this week. We built a command-line interface (CLI) automation agent using Python and Selenium/Selenium with an MCP-standard input/output (stdio) server to provide an automated way to submit Google Forms, along with structured execution and stability controls.

We finished preprocessing and fine-tuning GPT-2 on the Software Question dataset. After solving several encoding problems, tokenizing and splitting the software question dataset into appropriate tokens, and using the Hugging Face Trainer API to train the model, we were able to validate the model's literary generation performance.

We finished a prototype for our AI Mock Interview project, which included rubric-based scoring and adaptive difficulty to leverage large language models. We also began locally fine-tuning an open-source LLM using a QLoRA to reduce reliance on the API and to support an independent system. Additionally, we tested a fully offline Role-Wise Mock Interview System using semantically similar (SENC) solutions and a bidirectional long short-term memory (Bi-LSTM) model, with keyword matching for hybrid evaluation scoring.

Contribution

Md. Kamrul Islam (2211745642):

This week, I designed and implemented a simple automation agent using Python, Selenium, and an MCP stdio server. The CLI-based agent prompts the user for a Google Form URL, then triggers a custom MCP tool that launches Google Chrome, navigates to the provided URL, and submits the form by clicking the "Submit" button. The automation flow includes structured task execution, basic confirmation checking, and a clean browser shutdown to ensure stability. After completing each major task step, there is a 5-second pause.

Farhan Ishraque (2212002042):

I rode the waves of Software Questions.csv (dataset) loading and preprocessing, overcoming UnicodeDecodeError by using the Latin-1 encoding scheme. I created a text column combining Question and Answer for model input, then tokenized our data with a GPT-2 tokenizer. I took special care to maintain data integrity by paying attention to sequence length and the pad_token during the preprocessing. Finally, I converted the pandas DataFrame into a Hugging Face dataset.

After tokenizing the dataset, I split the data into training (80%) and test (20%) sets for model validation. I fine-tuned a pre-trained GPT2 using the HuggingFace Trainer API, providing

TrainingArguments to set the batch size, learning rate, evaluation strategy, etc., to optimize training. Finally, I saved the fine-tuned model and tokenizer, allowing them to be reused for future inference. Additionally, I tested the fine-tuned model's text-generation capabilities by providing a sample prompt and was pleased with how relevant the generated answers were. Used simple switching to use the Latin-1 encoding scheme to solve a UnicodeDecodeError last night when loading the initial data set.

Iram Shehzad (2131614642):

I have completed the AI Mock Interview component of my project. This was started with a prototype based on LLMs, where the system generates interview questions, scores users' answers to these questions using a rubric (relevance, structure, depth, and clarity), and provides feedback on user performance in JSON format. In addition, I built support for adaptive difficulty levels and can track individual sessions.

To further develop the machine learning component of this project and reduce reliance on external APIs, I decided to use a local fine-tuning approach. Currently, I am developing and training an open-source LLM in Google Colab and QLoRA (parameterisation-efficient fine-tuning). I also created a custom instruction dataset for evaluating mock interviews that is formatted for structured rubric-based scoring.

My next step for this project is to complete training, evaluate the performance of the fine-tuned LLM, and assess its integration into the system without the need for an API.

Atikul Islam Nahid (2211978042):

This week, I worked on this project, which presents a Machine Learning–based Role-Wise Mock Interview System. The system is designed to conduct mock interviews for various software engineering roles, including Frontend, Backend, DSA, and System Design. It selects questions based on the chosen role and automatically evaluates the candidate's written answers. To assess answer quality, a Bidirectional LSTM (Long Short-Term Memory) model is used to capture the semantic relationship between the question and the candidate's response. Text data is first tokenized and converted into embeddings, then processed through the BiLSTM network to generate a similarity score between 0 and 1. The final evaluation score is calculated using a hybrid approach that combines 70% semantic similarity from the LSTM model and 30% keyword coverage matching. This system provides a fully offline, customizable, and automated solution for mock interview evaluation, making it suitable for academic and research purposes

Completed Task:

- Various model training with the dataset.
- Create a Google Form for testing purposes.
- Automated Google Form submission.

Work in Progress

- Comparing various models' performance.
- Integrating a custom model into the mock interview feature.
- Read and write on a Google Form and submit it automatically.

Next Target

- Integrate the custom model into the mock interview feature.
- Fully read and write on a Google Form and submit it automatically.

Challenges

- Selecting and using the specific models for a specific task.
- Bypassing Google Chrome's automation blocking.
- Costly existing tools or websites in the current market.